Firebase

Technical guide to the CarePortal starter

A detailed, implementation-focused explanation of Firebase services, this project?s authentication flow, role-based dashboards, Firestore profile schema and rules, deployment, security boundaries, and troubleshooting.

What this guide covers

The current project uses Firebase Authentication (email/password and Google popup), Cloud Firestore (user profiles and roles), and initializes Cloud Storage. It is a starter application: the role dashboards are placeholders, not a finished clinical system.

Written against the current workspace files: src/firebase.js, src/main.js, src/styles.css, index.html, firestore.rules, .env.example, package.json, and README.md.

Contents

- 1. Firebase concepts and service map
- 2. Application architecture and project files
- 3. Create and configure the Firebase project
- 4. Firebase SDK initialization and environment configuration
- 5. Authentication flows: email/password and Google
- 6. User profile schema and transaction behavior
- 7. Role-based dashboard routing and authorization
- 8. Firestore Security Rules, explained
- 9. Bootstrap the first administrator
- 10. Cloud Storage: current state and safe next steps
- 11. Reliability, testing, and production hardening
- 12. Troubleshooting reference
- Appendix A. Current Firestore rules

A useful mental model: Authentication answers ?Who is signed in?? Firestore stores application records and the role used by this starter. Security Rules enforce what a client may access. Dashboard routing changes the interface, but the rules?not the interface?must guard data.

1. Firebase concepts and service map

Firebase is a set of managed Google Cloud services for applications. The client SDK communicates with configured services directly. This reduces the amount of backend infrastructure needed for common tasks, but it does not remove the need to design data, validate inputs, configure access rules, or operate the application responsibly.

Product	Purpose in an app
Firebase Authentication	Creates and signs in users; supplies a trusted UID and sign-in token to Firebase services.
Cloud Firestore	Document database used here for users/{uid} profile records and the application role.
Cloud Storage	Object/file storage for uploads. Initialized in this project, but no upload UI or Storage Rules are included.
Cloud Functions	Optional trusted server-side code; not configured in this starter. Useful for privileged workflows and event processing.
Hosting / Analytics / Messaging	Separate products; not configured here. The optional measurementId alone does not initialize Analytics.

Identity, data, and authorization are different concerns

- **Identity:** Firebase Authentication proves an account signed in and exposes request.auth.uid in rules.
- **Application profile:** this app stores role and display name in Firestore under a document ID equal to the Auth UID.
- **Authorization:** Firestore Rules evaluate each client read/write. Never rely only on hiding controls or rendering a particular dashboard.

The Firebase Web SDK config identifies the project and is delivered to browser code; it is not a server secret. Security Rules and careful handling of privileged credentials are the security boundary.

2. Application architecture and project files

File	Responsibility
index.html	Contains the sign-in/sign-up form and one dashboard shell for role-specific content.
src/firebase.js	Reads VITE_FIREBASE_* settings, validates required values, initializes one app, exports Auth, Firestore, and Storage instances.
src/main.js	Handles form state, email signup/login, Google popup, profile creation/read, auth-state changes, dashboard rendering, and sign-out.
src/styles.css	Responsive styling for the authentication screen and role dashboard.
firestore.rules	Controls access to users/{uid}; it does not yet protect future app data collections or Storage.
.env.example	Names the local Vite configuration variables. Copy to .env; do not publish actual local config files.
package.json	Declares firebase and Vite; npm run dev/build/preview scripts.
README.md	Project-specific setup, admin bootstrap, and security notes.

Runtime sequence

```
Browser loads index.html
  -> Vite loads src/main.js and src/styles.css
  -> src/firebase.js validates VITE_FIREBASE_* configuration
  -> initializeApp(config) creates the Firebase app
  -> getAuth(app), getFirestore(app), getStorage(app) expose services
  -> onAuthStateChanged(auth, callback) restores or observes identity
  -> app reads users/{uid} and renders the role-specific dashboard
```

The project uses JavaScript ES modules ("type": "module") and Firebase modular SDK imports. Import the shared service instances; do not initialize another Firebase app elsewhere.

3. Create and configure the Firebase project

1. Create a Firebase project in the Firebase Console, then register a Web app under Project settings -> General -> Your apps.

2. In Authentication -> Sign-in method, enable **Email/Password** and **Google**. Complete any provider setup requested by Firebase.

3. In Authentication -> Settings -> Authorized domains, ensure the local development hostname and deployed host are allowed. Google OAuth popups are blocked for unauthorized domains.

4. Create a Cloud Firestore database. Select an appropriate location before production use; location selection can affect latency and data residency.

5. Publish the included firestore.rules in Firestore Database -> Rules. The app profile transaction needs permission to create/read the signed-in user?s profile.

6. Copy .env.example to .env and populate it with the values from the web app SDK configuration. Restart Vite after changing environment variables.

7. From the firebaseSetup folder, install dependencies and start the app:

```
npm install
npm run dev
```

Vite prints a local URL. Production build: npm run build. Local preview of the build: npm run preview. Configure equivalent project variables in the hosting provider at build time.

Variable	What it identifies	Required by this code?
VITE_FIREBASE_API_KEY	Firebase client project API key	Yes
VITE_FIREBASE_AUTH_DOMAIN	Auth redirect/popup domain for project	Yes
VITE_FIREBASE_PROJECT_ID	Firebase project identifier	Yes
VITE_FIREBASE_STORAGE_BUCKET	Cloud Storage bucket name	Yes
VITE_FIREBASE_MESSAGING_SENDER_ID	Project messaging sender identifier	Yes
VITE_FIREBASE_APP_ID	Registered Firebase Web app identifier	Yes
VITE_FIREBASE_MEASUREMENT_ID	Analytics measurement identifier	No; optional and not used to initialize Analytics

Vite exposes variables prefixed with VITE_ to client code. Consequently these values are build-time configuration, not secrets. The project .gitignore now excludes .env and .env.* while allowing .env.example.

4. SDK initialization and environment configuration

In src/firebase.js, import.meta.env supplies values defined by Vite. The code checks required values and throws a descriptive error early if any are missing. measurementId is exempt from the required check. Then a single Firebase app instance is passed to each product factory.

```
const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
  measurementId: import.meta.env.VITE_FIREBASE_MEASUREMENT_ID,
};

export const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
export const storage = getStorage(app);
```

- initializeApp establishes the default Firebase app configuration for this JS runtime.
- getAuth, getFirestore, and getStorage create/use SDK service clients attached to that app. Their existence does not mean a provider, database rules, bucket, or application feature is configured.
- A missing configuration throws during module loading. This is deliberate: an explicit setup error is safer than a misleading ?connected? fallback.

5. Authentication flows

Email/password signup

```
createUserWithEmailAndPassword(auth, email.trim(), password)
  -> Firebase creates Auth account and signs the user in
  -> updateProfile(user, { displayName }) updates Auth display name
  -> ensureUserProfile(user, displayName) creates users/{uid} as patient
  -> onAuthStateChanged observes signed-in user and resolves profile
  -> showDashboard renders the patient dashboard
```

The name input is required in signup mode and is limited in the HTML to 100 characters. Password input has minlength=6. Firebase applies its own provider-side validation too. The frontend constraints improve usability but are not a security boundary.

Email/password login

```
signInWithEmailAndPassword(auth, email.trim(), password)
  -> Firebase validates credentials and changes Auth state
  -> onAuthStateChanged receives the authenticated user
  -> ensureUserProfile reads existing users/{uid}
  -> role is checked against admin | doctor | patient
  -> dashboard for that role is rendered
```

Google popup

```
const provider = new GoogleAuthProvider();
const result = await signInWithPopup(auth, provider);
await ensureUserProfile(result.user);
```

The same button handles first-time registration and later sign-in. Google returns an Auth user profile (including an email/display name where available); if there is no Firestore profile, the app creates one with role patient. The OAuth provider must be enabled and the domain authorized in Firebase Console.

Auth observer and persistent session

onAuthStateChanged is the app?s source of truth for whether an Auth user is currently signed in. It also handles restoring a previously persisted browser session after reload. For an authenticated user, the handler temporarily displays ?Loading your dashboard...? while reading/creating the Firestore profile. Signing out calls Firebase signOut(auth); the observer switches back to the auth screen.

Errors and user feedback

Known SDK error codes (invalid credentials, duplicate email, weak password, popup closed/blocked, unauthorized domain, and permission denied) are mapped to readable messages. Other errors fall back to the Firebase error message. A production app should avoid exposing internal details unnecessarily and should send operational diagnostics to a protected logging/monitoring system.

6. User profile schema and transaction behavior

A Firestore profile is stored at the document path users/{uid}, where uid is the Firebase Authentication UID. Matching document ID and UID makes direct self-profile checks straightforward in rules.

```
users/{uid} {
  uid: string,             // same value as the document ID / Auth UID
  email: string,           // Auth email when available
  displayName: string,     // signup name or Google/Auth display name
  role: "patient" | "doctor" | "admin",
  createdAt: timestamp    // server timestamp
}
```

ensureUserProfile uses runTransaction: it reads the profile, creates a missing profile with role patient and a server timestamp, or returns the existing profile. If a signup name is supplied and the stored name is empty, the transaction fills it. A transaction can be retried by Firestore when concurrent writes occur; keep its callback safe to rerun and do not perform external side effects inside it.

The helper intentionally does not overwrite an existing role with patient. This preserves a doctor/admin role when an existing user signs in through Google or the email form. Unknown role values are rejected instead of silently granting the default dashboard.

If Auth account creation succeeds but Firestore profile creation fails (for example, rules are unpublished), the user may exist in Authentication without a usable profile. The app shows the error; publish the rules and retry sign-in, after which a missing profile can be initialized as patient.

7. Role-based dashboard and authorization

Role	Dashboard content in starter
admin	Team access, people, portal settings placeholders.
doctor	My patients, appointments, care notes placeholders.
patient	My care team, appointments, personal information placeholders.

The roles object in src/main.js is a role-to-view mapping. After loading the profile, showDashboard selects the matching configuration and creates cards using DOM APIs and textContent. This avoids inserting profile-provided strings as HTML.

Important: this is **presentation routing**, not complete access control. A user can alter browser JavaScript or call Firebase APIs directly. The Firestore rules must authorize each data request. At present, rules protect only users/{userId}; all future data collections need their own scoped rules. No clinical records or patient-management workflow are implemented.

How privilege is granted

- Self-service signup cannot choose a role: client creates only a patient profile, and the rules also require role == "patient".
- A trusted project administrator bootstraps the first admin in the Firebase Console. This starter does not include a role-management screen or a server-side provisioning endpoint.
- The Firestore rules let an existing admin update only the role field of a profile to one of the three allowed values. Admins can grant or remove admin status, so keep admin accounts tightly controlled.
- When an admin changes a role in the Console, the open app does not subscribe to profile changes. The user can sign out/sign back in (or reload) to re-run profile lookup. A production app may use an explicit refresh or a profile listener.

8. Firestore Security Rules, explained

Rules are evaluated by Firestore on every client operation. They are not filters: a query is rejected if the rules cannot prove that every result is authorized. The current rules cover only the users collection; unmatched paths are denied by default.

Rule-by-rule summary

- signedIn() requires a non-null Auth context.

- isAdmin() reads the signed-in user?s profile and checks role == "admin". This document lookup is a dependent rules read and may be subject to Firestore rules-access-call limits and billing behavior.
- create requires the caller UID to equal the target document ID, allows only the expected profile keys, binds uid/email to Auth identity, validates displayName, forces role to patient, and requires createdAt to equal request.time (compatible with serverTimestamp).
- get permits a user to read their own profile, or an admin to read a profile. list permits only admins to query/list user profiles.
- update has two paths: owner may change only displayName; admin may change only role, and the new role must be in the allow-list. affectedKeys().hasOnly prevents unrelated fields being modified in that operation.
- delete is admin-only. Production systems should carefully consider whether profile deletion should exist and how it relates to Auth account deletion and data-retention requirements.

Deploy the rules in Firebase Console -> Firestore Database -> Rules. Keep the checked-in rules in sync with the deployed rules. For repeatable deployments, initialize Firebase CLI configuration and use firebase deploy --only firestore:rules.

9. Bootstrap the first administrator

1. Create your account in the app. It should be created as patient.
2. In Firebase Console -> Authentication -> Users, locate the account and copy its UID.
3. In Firestore Database -> Data, open users, then the document whose ID is exactly that UID.
4. Change the field role to the string admin (not a map/object). Save the document.
5. Sign out and sign in again or reload, so the app reads the updated profile and renders the admin dashboard.

Console access is privileged and can edit data outside client-side rules. Limit Firebase project owner/editor roles to trusted operators and use multi-factor authentication for project administrators. Do not implement a browser-side ?make me admin? button.

10. Cloud Storage: current state and safe next steps

src/firebase.js creates a Storage client and exports storage. The application currently has no upload control, no Storage SDK operation, and no Storage Security Rules file. Initialization alone does not create safe upload behavior or grant permissions.

```
import { storage } from "./firebase.js";
import { ref, uploadBytes, getDownloadURL } from "firebase/storage";

const objectRef = ref(storage, `uploads/${user.uid}/${safeFileName}`);
await uploadBytes(objectRef, file);
const downloadUrl = await getDownloadURL(objectRef);
```

Before implementing uploads: create/confirm the bucket; write and publish Storage Rules; authenticate the user; use UID-scoped paths; validate size and content type; avoid trusting a filename; decide whether a download URL should be shareable; and decide how deletion/retention works. Firestore Rules do not protect Storage. Healthcare or other sensitive documents require a dedicated privacy, compliance, retention, and access-control design before production use.

11. Reliability, testing, and production hardening

Recommended functional test matrix

Scenario	Expected result
Fresh email signup	Auth user created; Firestore users/{uid} created with patient role; patient dashboard shown.
Existing email signup	Readable duplicate-account message; no second profile.
Valid email login	Existing profile role determines dashboard.
Wrong password	Sign-in fails and user receives a readable error.
First-time Google popup	Auth account created; missing profile becomes patient.
Existing Google account	Existing Firestore role is preserved.
User reads own profile	Allowed by rules.
User reads another profile	Denied unless caller is admin.
Patient changes own role	Denied.
Patient changes own displayName	Allowed.
Admin changes role to valid role	Allowed.
Admin writes invalid role or extra fields with role update	Denied.
No Firestore profile / rules not published	Profile creation fails; error is shown.

Before production

- Write and test Firestore rules for every application collection; write separate Storage Rules. Consider Firebase Emulator Suite for repeatable local Auth/Firestore/rules tests.
- Add a real admin management workflow with explicit server-side authorization if role assignment needs to be done inside the product. A trusted backend using Admin SDK must never expose its service account to the browser.
- Add loading, retry, empty, and error states. Consider account recovery, email verification, password reset, account linking, and popup fallback behavior.
- Define data minimization, audit logging, backups, retention/deletion, operational monitoring, and incident response. Do not use this placeholder as a medical-record system without a full security and compliance review.
- Use separate Firebase projects for development, staging, and production, with appropriately restricted operators and rules.
- Run npm run build for a production compile. Review dependency audit warnings independently; do not blindly force dependency upgrades.

12. Troubleshooting reference

Symptom	Likely checks
?Missing Firebase config values?	Confirm .env is in firebaseSetup, variable names start VITE_FIREBASE_, values are filled in, and Vite was restarted. Do not paste private server credentials.
auth/operation-not-allowed	Enable Email/Password or Google in Authentication -> Sign-in method.
auth/unauthorized-domain	Add the current host to Authentication -> Settings -> Authorized domains.
Popup blocked/closed	Allow popups; test in a normal browser window; verify Google provider configuration and authorized domain.
permission-denied reading/creating profile	Publish firestore.rules to the same Firebase project ID used by .env; inspect Firestore Rules simulator/logs.
Profile created but no dashboard	Check users/{Auth UID} exists and role is exactly admin, doctor, or patient; reload or sign out/in.
Dashboard changes but data remains accessible	UI routing is not authorization. Create rules for the data collection; inspect direct client requests with Emulator/rules tests.
Storage upload denied	This starter has no Storage Rules. Configure the bucket and publish a separate, least-privilege Storage ruleset.
Build succeeds but browser uses wrong project	Check environment variables at build time and ensure the deployed artifact was built for the intended project.

When reporting an issue, record the Firebase error code and which step failed, but do not share passwords, OAuth tokens, service-account files, or sensitive user data.

Appendix A. Current Firestore rules

This appendix reproduces the project?s rules. Review and test them before deploying; rules should evolve with the data model.

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() {
      return request.auth != null;
    }

    function isAdmin() {
      return signedIn()
        && get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role == "admin";
    }

    function validRole(role) {
      return role in ["admin", "doctor", "patient"];
    }

    match /users/{userId} {
      allow create: if signedIn()
        && request.auth.uid == userId
        && request.resource.data.keys().hasOnly([
          "uid", "email", "displayName", "role", "createdAt"
        ])
        && request.resource.data.uid == userId
        && request.resource.data.email == request.auth.token.email
        && request.resource.data.displayName is string
        && request.resource.data.role == "patient"
        && request.resource.data.createdAt == request.time;

      allow get: if signedIn() && (request.auth.uid == userId || isAdmin());
      allow list: if isAdmin();

      allow update: if signedIn()
        && (
          (request.auth.uid == userId
            && request.resource.data.diff(resource.data).affectedKeys()
                 .hasOnly(["displayName"])
            && request.resource.data.displayName is string)
          || (isAdmin()
            && request.resource.data.diff(resource.data).affectedKeys()
                 .hasOnly(["role"])
            && validRole(request.resource.data.role))
        );

      allow delete: if isAdmin();
    }
  }
}
```

Firebase documentation: firebase.google.com/docs | Console: console.firebase.google.com.